Hướng dẫn triển khai và khôi phục mô phỏng Truyền đặc trưng ngữ nghĩa có xét nhầm lẫn lớp qua kênh vô tuyến

Tài liệu kỹ thuật cho nhóm sinh viên nghiên cứu khoa học

Tóm tắt nội dung

Tài liệu này hướng dẫn triển khai gói mã mô phỏng cho hệ thống truyền đặc trưng ngữ nghĩa phục vụ phân loại bệnh lá cà chua. Quy trình được tổ chức theo các khối: trích xuất đặc trưng tại phía phát, ước lượng cặp lớp cạnh tranh, xây dựng đồ thị nhầm lẫn phụ thuộc SNR, xếp hạng nhóm đặc trưng theo biên quyết định cặp lớp, phân bổ bảo vệ dưới ngân sách symbol cố định, và phân loại tại phía thu. Mỗi giai đoạn có checkpoint, tệp trạng thái và tiêu chí đạt. Khi mất điện hoặc tiến trình bị dừng, sinh viên chạy lại đúng lệnh trước đó; chương trình tự nạp checkpoint gần nhất và bỏ qua các ô mô phỏng đã hoàn thành.

Index Terms

Semantic communication, task-oriented communication, feature transmission, class confusion, unequal protection, checkpoint, AWGN, Rayleigh fading.

I. Mục tiêu và phạm vi hiện thực

Trong kịch bản camera biên gửi thông tin ảnh lá cà chua tới máy chủ, đầu ra cần bảo toàn là nhãn bệnh và độ tin cậy phân loại, không phải ảnh khôi phục có PSNR cao. Gói mã hiện thực đường truyền đặc trưng M1–M5 của bản thảo: MobileNetV3-Small hoặc ResNet-18 tại phía phát, bộ ước lượng hai lớp cạnh tranh, codebook đặc trưng theo biên logit, bộ ánh xạ đặc trưng sang symbol phức, kênh AWGN/Rayleigh, bộ khôi phục nhóm đặc trưng và bộ phân loại phía thu.

Mục tiêu kiểm chứng là: với cùng ngân sách symbol và công suất trung bình, phương án đề xuất có cải thiện $F_{1,\mathrm{macro}}$, recall lớp yếu nhất hoặc mức nhầm lẫn cặp lớp xấu nhất so với global importance, SNR-only và pairwise-only hay không. Kết quả không được quy cho phương pháp nếu ngân sách symbol khác nhau.

Phiên bản mã này khóa phần đóng góp chính và các ablation trực tiếp. Chuỗi JPEG/BPG–channel coding–QPSK, DeepJSCC tái tạo ảnh, OFDM/TDL và LDPC chưa được trộn vào pipeline lõi. Các baseline này phải được bổ sung sau khi cơ chế confusion-aware vượt qua kiểm tra nhân quả ở AWGN/Rayleigh; cách tách này tránh việc một modem hoặc decoder phức tạp che khuất tác động của bộ chọn đặc trưng.

II. Cấu trúc thư mục

```
confusion_aware_semcom_code/
|-- configs/
| |-- default.yaml
| `-- smoke.yaml
|-- src/semcom/
| |-- data.py, models.py, channel.py
| |-- policy.py, losses.py, metrics.py
| |-- checkpoint.py, engine.py
| `-- config.py, io.py, seed.py
|-- scripts/
```

```
| |-- prepare_dataset.py
| |-- train_clean.py
| |-- build_artifacts.py
| |-- train_semcom.py
| |-- evaluate.py
| |-- analyze_artifacts.py
| |-- plot_results.py
| |-- verify_checkpoints.py
| |-- run_pipeline.py
| `-- smoke_test.py
`-- guide/
```

Các tệp trong `src/semcom` chứa mô hình và logic khoa học. Các tệp trong `scripts` chỉ điều phối thí nghiệm, không thay đổi định nghĩa phương pháp.

III. Mô hình được hiện thực

A. Đặc trưng và nhóm đặc trưng

Ảnh đầu vào $\boldsymbol{x}$ được ánh xạ thành vector $\boldsymbol{z} \in \mathbb{R}^D$ và chia thành G nhóm:

$$\boldsymbol{z} = f_\theta(\boldsymbol{x}), \qquad \boldsymbol{Z} = \text{reshape}(\boldsymbol{z}) \in \mathbb{R}^{G \times d_g}, \quad D = G d_g. \tag{1}$$

Phép chia nhóm quyết định đơn vị chọn và bảo vệ trên đường truyền; cấu hình mặc định dùng $D = 256$, $G = 64$, và $d_g = 4$.

B. Đồ thị nhầm lẫn phụ thuộc SNR

Tại mỗi mức SNR γ, trọng số giữa hai lớp được tính từ confusion matrix trên tập validation:

$$w_{ij}(\gamma) = \frac{1}{2}\left[\Pr(\hat{y} = j \mid y = i, \gamma) + \Pr(\hat{y} = i \mid y = j, \gamma)\right]. \tag{2}$$

Đại lượng này điều khiển mức độ nghiêm trọng của cặp lớp tại từng điều kiện kênh; nó không được ước lượng trên tập test.

C. Xếp hạng đặc trưng theo biên cặp lớp

Bộ phân loại cục bộ là lớp tuyến tính trên $\boldsymbol{z}$. Với cặp (i, j), score của chiều đặc trưng k là

$$s_k^{(i,j)}(\boldsymbol{x}) = \left| z_k \frac{\partial (a_i - a_j)}{\partial z_k} \right|. \tag{3}$$

Do local head tuyến tính, gradient bằng hiệu hai vector trọng số. Codebook được tính offline bằng trung bình $|z_k|\,|w_{i,k} - w_{j,k}|$ trên validation set, sau đó cộng theo từng nhóm. Cách này bảo toàn đúng định nghĩa gradient–activation nhưng tránh tính backward cho từng mẫu khi triển khai.

D. Ngân sách slot và số symbol

Mỗi nhóm có $d_g/2$ symbol phức. Nếu ngân sách là B_s slot nhóm thì

$$N_{\text{sym}} = B_s \frac{d_g}{2}. \tag{4}$$

Phương trình này khóa tài nguyên kênh của mọi phương án. Khi chỉ chọn $K < B_s$ nhóm, các slot còn lại lặp các nhóm có thứ hạng cao; phía thu cân bằng kênh và trung bình các bản lặp.

E. Kênh truyền

Kênh AWGN và Rayleigh block fading được mô phỏng theo

$$\boldsymbol{r} = \boldsymbol{u} + \boldsymbol{n}, \qquad \text{AWGN}, \tag{5}$$

$$\boldsymbol{r} = h\boldsymbol{u} + \boldsymbol{n}, \qquad h \sim \mathcal{CN}(0,1). \tag{6}$$

Nhiễu phức được đặt theo $\sigma_n^2 = P10^{-\gamma/10}$. Hệ số chuẩn hóa đặc trưng là một hằng số được hiệu chuẩn trên tập train và dùng chung ở hai phía; không sử dụng hệ số chuẩn hóa riêng từng mẫu nên không tạo kênh báo hiệu phụ không được tính.

IV. Chuẩn bị môi trường

```
cd confusion_aware_semcom_code
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt
python scripts/smoke_test.py
```

Checkpoint C0 đạt khi lệnh smoke test kết thúc bằng `Smoke test passed` và sinh `summary_aggregated.csv`. Nếu C0 không đạt, không chạy dữ liệu thật.

V. Chuẩn bị dữ liệu

Dữ liệu phải có cấu trúc ImageFolder:

```
PlantVillage/
|-- Tomato___Bacterial_spot/
|-- Tomato___Early_blight/
|-- Tomato___Late_blight/
`-- ...
```

Sửa ba trường trong `configs/default.yaml`: `data.root`, `data.manifest_dir`, và `project.outpu`
Nếu tên lớp chứa từ "Tomato", giữ `class_name_contains: Tomato`; nếu không, điền chính xác danh sách `selected_classes`.

Chạy:

```
python scripts/prepare_dataset.py --config configs/default.yaml
```

Chương trình chia dữ liệu theo source identifier trước khi augmentation. Các ảnh có hậu tố phổ biến như `_aug`, `_flip`, `_rot` được giữ trong cùng một split để giảm nguy cơ rò rỉ dữ liệu.

Checkpoint C1 đạt khi:

- `train.csv`, `val.csv`, `test.csv` tồn tại;
- số lớp và số mẫu từng lớp trong `metadata.json` đúng với dữ liệu;
- không có source identifier xuất hiện ở nhiều split;
- mỗi lớp có mẫu ở validation và test.

VI. Chạy pipeline và tự khôi phục

Lệnh đầy đủ cho một seed:

```
python scripts/run_pipeline.py \
  --config configs/default.yaml \
  --seed 11
```

Khi mất điện hoặc tiến trình bị dừng, chạy lại đúng lệnh trên. Không thêm `--force`. Pipeline tự thực hiện:
1) nạp `latest.pt` của giai đoạn đang huấn luyện;
2) khôi phục model, optimizer, scheduler, AMP scaler và trạng thái RNG;
3) đọc `pipeline_state.json`;
4) bỏ qua artifact và mô hình đã có `best.pt`;
5) bỏ qua từng ô kết quả JSON đã hoàn thành.

Checkpoint được ghi nguyên tử qua file tạm rồi đổi tên. Nếu mất điện đúng lúc ghi, phiên bản checkpoint trước vẫn còn hợp lệ. Với `save_every_steps=300`, lượng tính toán mất tối đa xấp xỉ 299 batch. Có thể giảm khoảng này khi nguồn điện không ổn định, đổi lại số lần ghi đĩa tăng.

Kiểm tra toàn bộ checkpoint sau sự cố hoặc sau khi chép dữ liệu:

```
python scripts/verify_checkpoints.py \
  --config configs/default.yaml --seed 11
```

Không đổi config giữa hai lần resume. Hash cấu hình được lưu trong checkpoint. Nếu thay đổi tham số, chương trình tạo thư mục thí nghiệm khác hoặc từ chối nạp checkpoint không tương thích.

VII. Các giai đoạn và tiêu chí đạt

A. Giai đoạn A: bộ phân loại không qua kênh

```
python scripts/train_clean.py \
  --config configs/default.yaml --seed 11
```

Đầu ra chính là `checkpoints/clean/best.pt` và `clean_history.csv`. Không chuyển sang communication stage nếu mô hình clean chưa hội tụ hoặc recall một lớp gần bằng không.

Checkpoint C2 đạt khi:
- training loss giảm ổn định;
- validation macro-F1 không còn tăng rõ trong nhiều epoch cuối;
- confusion matrix clean chỉ ra các cặp lớp khó hợp lý;
- số tham số và backbone đúng cấu hình.

B. Giai đoạn B: artifact cố định

```
python scripts/build_artifacts.py \
  --config configs/default.yaml --seed 11
python scripts/analyze_artifacts.py \
  --config configs/default.yaml --seed 11
```

Ba artifact bắt buộc là `feature_scale.pt`, `confusion_graph.pt`, và `pairwise_codebook.pt`.

Checkpoint C3 đạt khi:
- hệ số chuẩn hóa hữu hạn và dương;
- top confusion pair không hoàn toàn ngẫu nhiên giữa các SNR;
- ít nhất một cặp lớp có trọng số nhầm lẫn tăng khi SNR giảm;
- top feature groups của các cặp lớp không đồng nhất hoàn toàn.

Nếu thứ tự cặp lớp không thay đổi theo SNR, đóng góp “SNR-conditioned confusion graph” chưa được chứng minh. Khi đó phải giảm số lớp, kiểm tra classifier, hoặc xem lại cách hiệu chuẩn đường truyền full-feature trước khi viết claim.

C. Giai đoạn C: huấn luyện từng phương án

Ví dụ chạy phương án đề xuất với 32 slot:

```
python scripts/train_semcom.py \
  --config configs/default.yaml \
  --seed 11 \
  --scheme proposed \
  --budget-slots 32
```

Các scheme được hỗ trợ:

Scheme	Vai trò kỹ thuật
`random`	Chọn ngẫu nhiên; kiểm tra lợi ích của ranking.
`global`	Một ranking chung cho toàn bộ lớp; baseline importance-aware.
`class_specific`	Ranking theo lớp top-1, chưa dùng cặp lớp.
`snr_only`	Ranking global, số nhóm thay đổi theo SNR.
`pairwise`	Ranking cặp lớp và keep ratio theo SNR, không điều chỉnh bằng trọng số confusion.
`proposed`	Ranking cặp lớp kết hợp confusion severity và lặp bảo vệ dưới cùng budget.
`full`	Gửi toàn bộ nhóm; chỉ hợp lệ khi budget slot không nhỏ hơn số nhóm.

Checkpoint C4 đạt khi `best.pt` tồn tại cho từng tổ hợp scheme–budget, validation macro-F1 hữu hạn, và công suất phát trung bình gần giá trị cấu hình. Nếu một scheme dùng ít hoặc nhiều symbol hơn, phải sửa policy trước khi so sánh.

D. Giai đoạn D: đánh giá Monte Carlo

```
python scripts/evaluate.py \
  --config configs/default.yaml --seed 11
python scripts/plot_results.py \
  --config configs/default.yaml --seed 11 --channel awgn
```

Mỗi ô $(\text{scheme}, B_s, \text{channel}, \text{SNR}, \text{MC seed})$ được lưu thành một JSON. Khi lỗi giữa chừng, chạy lại lệnh; các JSON đã có không được tính lại.

Checkpoint C5 đạt khi có đủ các cột: accuracy, macro-F1, minimum-class recall, worst directed confusion, complex symbols/sample, selected groups, repetition ratio, runtime và top-2 pair coverage.

VIII. Thiết kế mô phỏng theo thứ tự

Không chạy toàn bộ ma trận ngay từ đầu. Thực hiện theo ba vòng.

A. Vòng 1: kiểm tra cơ chế

Dùng một seed, AWGN, budget 32, ba scheme `global`, `pairwise`, `proposed`, và SNR $\{0, 5, 10\}$ dB. Mục tiêu là xác nhận code không lỗi và policy thực sự thay đổi selected groups hoặc repetition ratio.

B. Vòng 2: ablation nhân quả

Khóa một budget chính, chạy năm seed trên AWGN. So sánh:

1) global với pairwise để tách tác động của pair-specific ranking;
2) pairwise với proposed để tách tác động của confusion severity;
3) snr-only với proposed để tách tác động của class-pair information;
4) zero filling với mask token;
5) predicted top-2 pair với oracle pair chỉ như upper bound.

Mã hiện tại không dùng oracle label trong policy triển khai. Nếu bổ sung oracle ablation, phải đặt tên rõ là upper bound và không đưa vào phương án khả thi.

C. Vòng 3: độ bền kênh và tài nguyên

Sau khi Vòng 2 có hiệu ứng rõ, chạy AWGN và Rayleigh trên SNR $\{-5, 0, 5, 10, 15, 20\}$ dB, budget $\{8, 16, 32, 64\}$, năm seed. CSI error chỉ được bật sau khi perfect-CSI result ổn định.

IX. Kết quả phải xuất cho bài báo

Bộ hình tối thiểu gồm:

- macro-F1 theo SNR tại hai budget cố định;
- minimum-class recall theo SNR;
- macro-F1 theo số symbol tại một SNR thấp và một SNR trung bình;
- worst-pair confusion theo SNR;
- confusion matrix của clean, global và proposed;
- runtime–performance của offline codebook và online scoring nếu có.

Bảng chính phải báo cáo cùng lúc metric tác vụ, tài nguyên và độ phức tạp. Không lập bảng chỉ có accuracy.

X. Quy tắc kết luận

Phương án chỉ được mô tả là *confusion-aware* nếu proposed vượt pairwise-only dưới cùng N_{sym}. Chỉ được mô tả là *channel-adaptive* nếu mask, số nhóm hoặc số bản lặp thay đổi theo SNR và ablation SNR-only đã được báo cáo. Chỉ được mô tả là *class-wise reliable* nếu minimum-class recall hoặc worst-pair confusion cải thiện với khoảng tin cậy trên nhiều seed.

Quan sát kết quả theo nhịp: quan sát $\rightarrow$ so sánh $\rightarrow$ giải thích $\rightarrow$ hàm ý. Ví dụ: “Tại 0–5 dB, proposed tăng minimum-class recall so với pairwise-only dưới cùng 64 complex symbols; mức tăng đi cùng số nhóm chọn thấp hơn và nhiều bản lặp hơn, cho thấy confusion severity đã chuyển tài nguyên từ độ rộng representation sang bảo vệ cặp lớp khó. Ở 15–20 dB, các phương án hội tụ vì lỗi kênh không còn là bottleneck.” Chỉ dùng câu này khi số liệu thực sự hỗ trợ.

XI. Xử lý lỗi thường gặp

Hiện tượng	Cách xử lý
Config/checkpoint mismatch	Dùng `resolved_config.yaml` trong run directory; không ép nạp checkpoint của cấu hình khác.
CUDA hết bộ nhớ	Giảm batch size; không đổi feature dimension giữa lúc resume. Sau khi đổi config, chạy run mới.
DataLoader treo	Đặt `num_workers: 0`, `persistent_workers: false`.
Mất điện giữa epoch	Chạy lại đúng lệnh; `latest.pt` hoặc `emergency.pt` được nạp tự động.
Một JSON kết quả lỗi	Xóa đúng JSON hỏng, chạy lại evaluate; không dùng `--force` cho toàn bộ lưới.
Full scheme báo thiếu slot	Chỉ chạy full khi `budget_slots >= num_groups`.
Macro-F1 cao nhưng min recall thấp	Kiểm tra mất cân bằng lớp, pair estimator và λ_f; không kết luận reliability.
Proposed giống pairwise	Confusion thresholds không kích hoạt hoặc graph không phân tách severity; kiểm tra `confusion_edges_top.csv`.

XII. Lệnh chạy nhiều seed

Sau khi khóa `default.yaml`, chạy độc lập từng seed:

```
for s in 11 22 33 44 55; do
  python scripts/run_pipeline.py \
    --config configs/default.yaml --seed $s
done
```

Không gộp checkpoint giữa các seed. Mỗi seed có thư mục riêng và trạng thái RNG riêng. Sau khi đủ năm seed, tổng hợp mean, standard deviation và 95% confidence interval từ các tệp `summary_raw.csv`.

XIII. Kết luận vận hành

Trình tự bắt buộc là C0 $\rightarrow$ C1 $\rightarrow$ C2 $\rightarrow$ C3 $\rightarrow$ C4 $\rightarrow$ C5. Không chuyển sang Rayleigh, CSI error, digital modulation hoặc PlantDoc trước khi cơ chế proposed vượt các ablation trực tiếp ở AWGN dưới cùng ngân sách symbol. Mục tiêu của mã không phải tạo đường cong đẹp, mà xác định chính xác điều kiện SNR và budget mà confusion-aware protection có tác động đo được lên đầu ra phân loại phía thu.